



数据库系统

第九章 查询处理和查询优化

胡金鹏



- **掌握**

- RDBMS的查询处理的任务和工作步骤
- 查询操作的主要方法

- **了解理解**

- 利用等价变换规则和优化策略对关系代数表达式进行优化。
- 关系代数表达式的优化算法、优化过程，用语法树表示



查询处理和查询优化



9.0 概述

9.1 查询处理

9.2 查询优化

9.3 代数优化

9.4 物理优化

9.5 小结



Codd自1970年起即对关系数据库的原理与理论作了详细的探讨与研究，但是，在70年代关系数据库始终无法走向商品化，其主要原因就是它的**查询效率低下**，这影响了它的实用性

为解决问题，在关系数据库的发展历史上又继续奋斗了近十年，在近十年的研究中发现了关系数据库理论的很多优点，其最为引人注目的一点**即：**

关系查询表达式在关系式理论中可以有**多种等价**表达形式，而不同表达形式的**查询效率**又有明显的不同



(1)关系代数的**有效的等价**表达式；

(2)不同表达**形式与查询效率**间的必然联系；

(3)获取**较高查询效率**表达式的算法。

-----这些结果为提高查询效率提供了有效的方法与手段



查询处理和查询优化



9.0 概述

9.1 查询处理

9.2 查询优化

9.3 代数优化

9.4 物理优化

9.5 小结

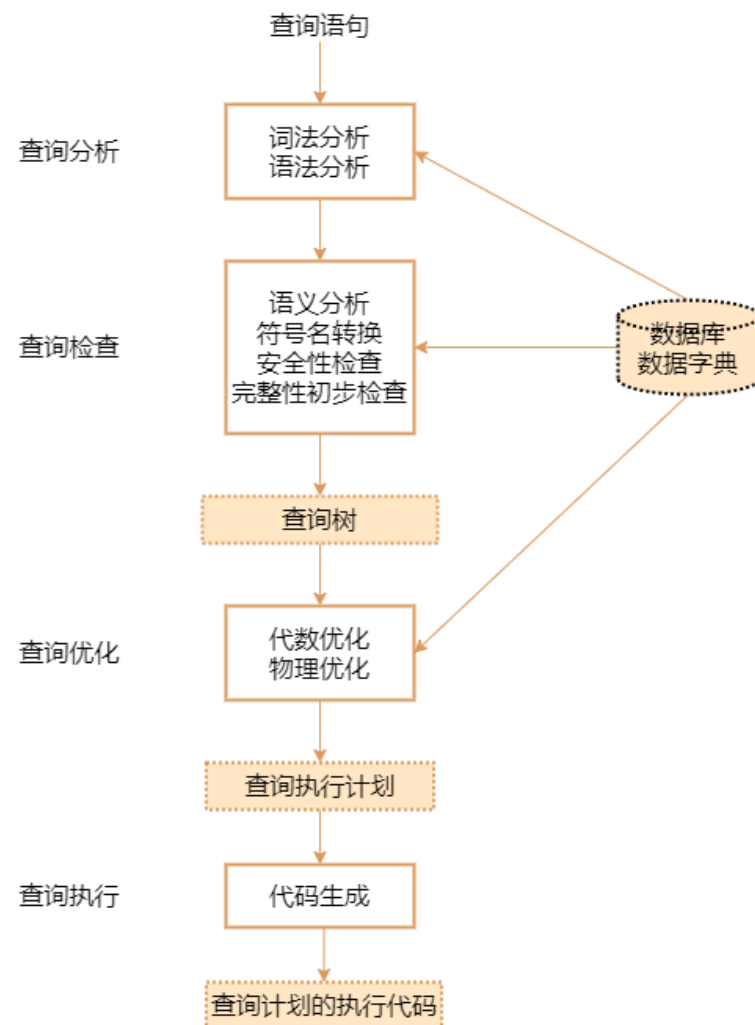


- 1. 查询处理的任务

把用户提交给RDBMS的查询语句
转换为高效的执行计划。

- 2. 基本步骤 Basic Steps in Query Processing

- 1. 查询分析
- 2. 查询检查
- 3. 查询优化
- 4. 查询执行





- **1.查询分析**
 - 对查询语句**进行扫描、词法分析和语法分析**
 - 词法分析：从查询语句中识别出语言符号
 - 语法分析：进行语法检查



• 2.查询检查

(1) 合法性检查：根据数据字典有关的模式定义检查语句中的数据库对象，如关系名、属性名是否存在和有效。

(2) 视图转换：如果是对视图的操作，则要用视图消解方法把对视图的操作转换成对基本表的操作。

(3) 安全性和完整性初步检查：根据数据字典中的用户权限和完整性约束定义对用户的存取权限进行检查。

➤ 检查通过后把SQL语句转换为内部表示，即**等价的关系代数表达式**。关系数据库管理系统一般都用**查询树**，也称**语法分析树**来表示扩展的关系代数表达式。



- **3.查询优化：每个查询都有很多种方案，选择一个高效执行的查询处理策略**
 - 利用数据库目录中的统计信息来估计代价
 - 如每个关系的元组数, 元组大小等.
 - 查询优化分类：
 - 代数优化：指关系代数表达式的优化
 - 物理优化：指存取路径和底层操作算法的选择
 - 查询优化方法选择的依据：
 - 基于规则(rule based)
 - 基于代价(cost based)
 - 基于语义(semantic based)



- **4.查询执行**

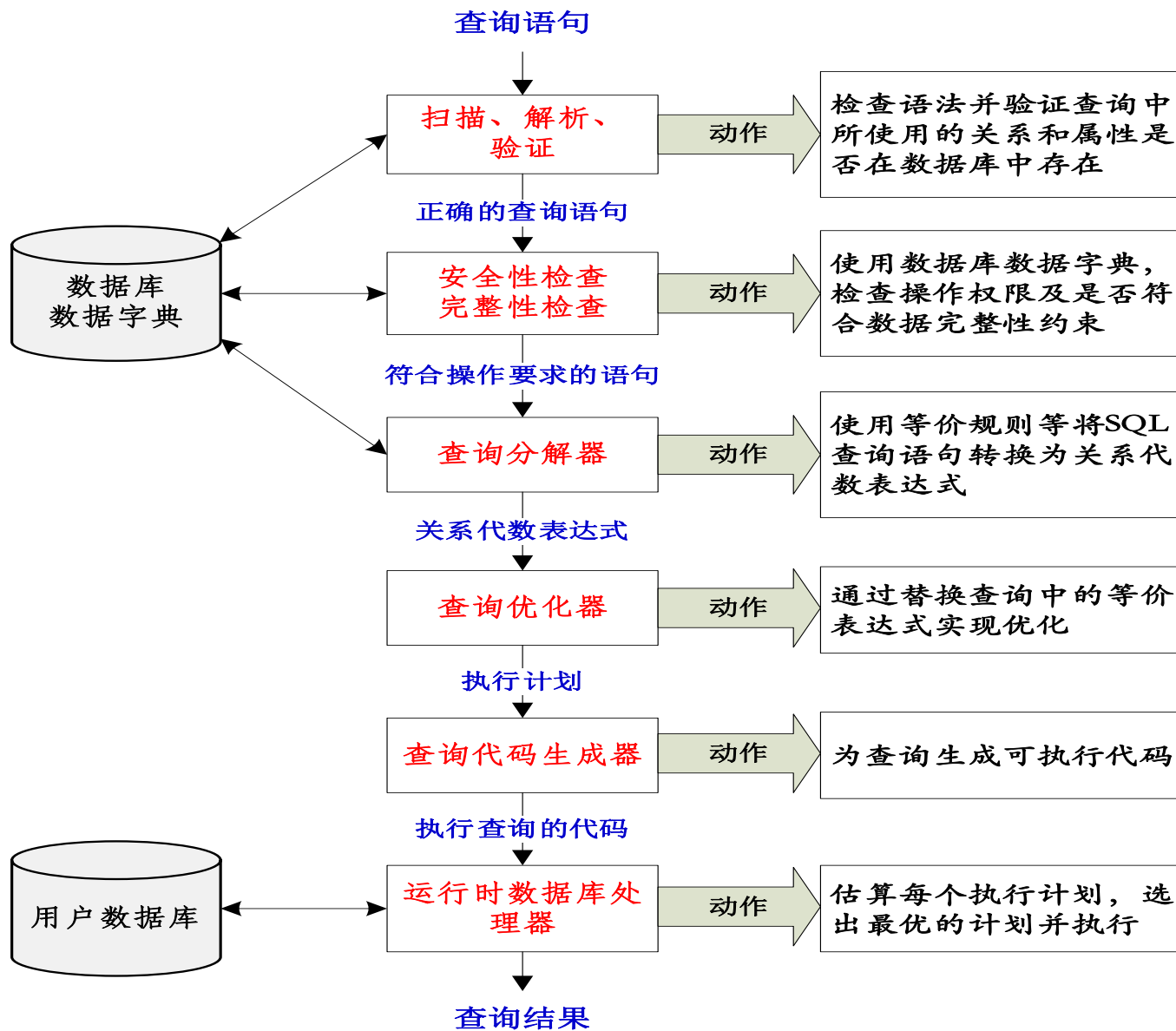
- 依据优化器得到的执行策略生成查询计划
- 代码生成器(code generator)生成执行查询计划的代码



查询处理和查询优化



查询处理的典型过程





实现查询操作的算法示例

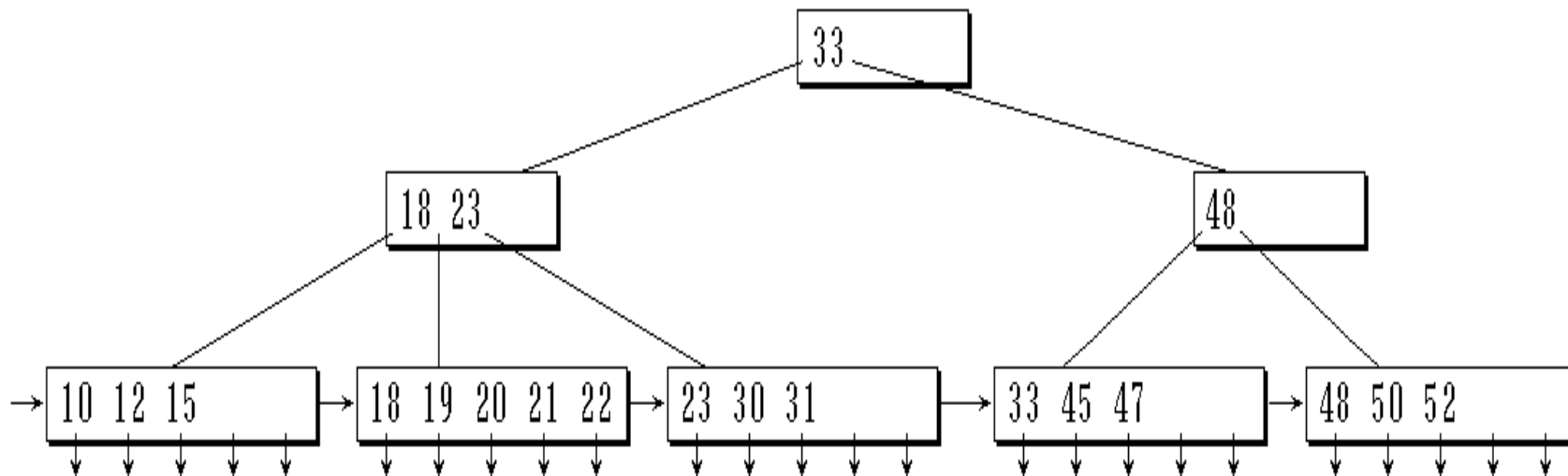
• 一、选择操作的实现

1. 简单的全表扫描方法

- 对查询的基本表顺序扫描，逐一检查每个元组是否满足选择条件，把满足条件的元组作为结果输出
- 适合小表，不适合大表

2. 索引(或散列)扫描方法

- 适合选择条件中的属性上有索引(例如B+树索引或Hash索引)
- 通过索引先找到满足条件的元组主码或元组指针，再通过元组指针直接在查询的基本表中找到元组



- 散列方法是在表项的存储位置与它的关键码之间建立一个确定的对应函数关系 $Hash()$ ，使每个关键码与结构中一个唯一存储位置相对应：

$$Address = Hash (Rec.key)$$

- 在存放表项时，依相同函数计算存储位置，并按此位置存放。这种方法就是**散列方法**。在散列方法中使用的转换函数叫做**散列函数**。而按此种想法构造出来的表或结构就叫做**散列表**。
- 使用散列方法进行搜索不必进行多次关键码的比较，搜索速度比较快，可以直接到达或逼近具有此关键码的表项的实际存放地址。



【例】 `Select * from student where <条件表达式> ;`

考虑<条件表达式>的几种情况：

C1 : 无条件 ;

C2 : Sno = '200215121' ;

C3 : Sage > 20 ;

C4 : Sdept = 'CS' AND Sage > 20 ;



(1) 全表扫描算法

假设可以使用的内存为M块，全表扫描算法思想：

- ① 按照物理次序读Student的M块到内存
- ② 检查内存的每个元组t，如果满足选择条件，则输出t
- ③ 如果student还有其他块未被处理，重复①和②



(2) 索引扫描算法

【例-C2】以C2为例，Sno = '200215121'，并且Sno上有索引(或Sno是散列码)

- **使用索引(或散列)得到Sno为 '200215121' 元组的指针**
- **通过元组指针在student表中检索到该学生**



(2) 索引扫描算法

【例-C3】 以C3为例， $Sage > 20$ ，并且Sage 上有B+树索引

- 使用B+树索引找到 $Sage = 20$ 的索引项，以此为入口点在B+树的顺序集上得到 $Sage > 20$ 的所有元组指针
- 通过这些元组指针到student表中检索到所有年龄大于20的学生。



【例C-4】 以C4为例， $Sdept = 'CS' \text{ AND } Sage > 20$ ，如果Sdept和Sage上都有索引：

算法一： 分别用上面两种方法分别找到 $Sdept = 'CS'$ 的一组元组指针和 $Sage > 20$ 的另一组元组指针

- 求这2组指针的交集
- 到student表中检索
- 得到计算机系年龄大于20的学生

算法二： 找到 $Sdept = 'CS'$ 的一组元组指针，

- 通过这些元组指针到student表中检索
- 对得到的元组检查另一些选择条件(如 $Sage > 20$)是否满足
- 把满足条件的元组作为结果输出。



实现查询操作的算法示例

• 二、 连接操作的实现

连接操作是查询处理中最耗时的操作之一，本节只讨论等值连接（或自然连接）最常用的实现算法

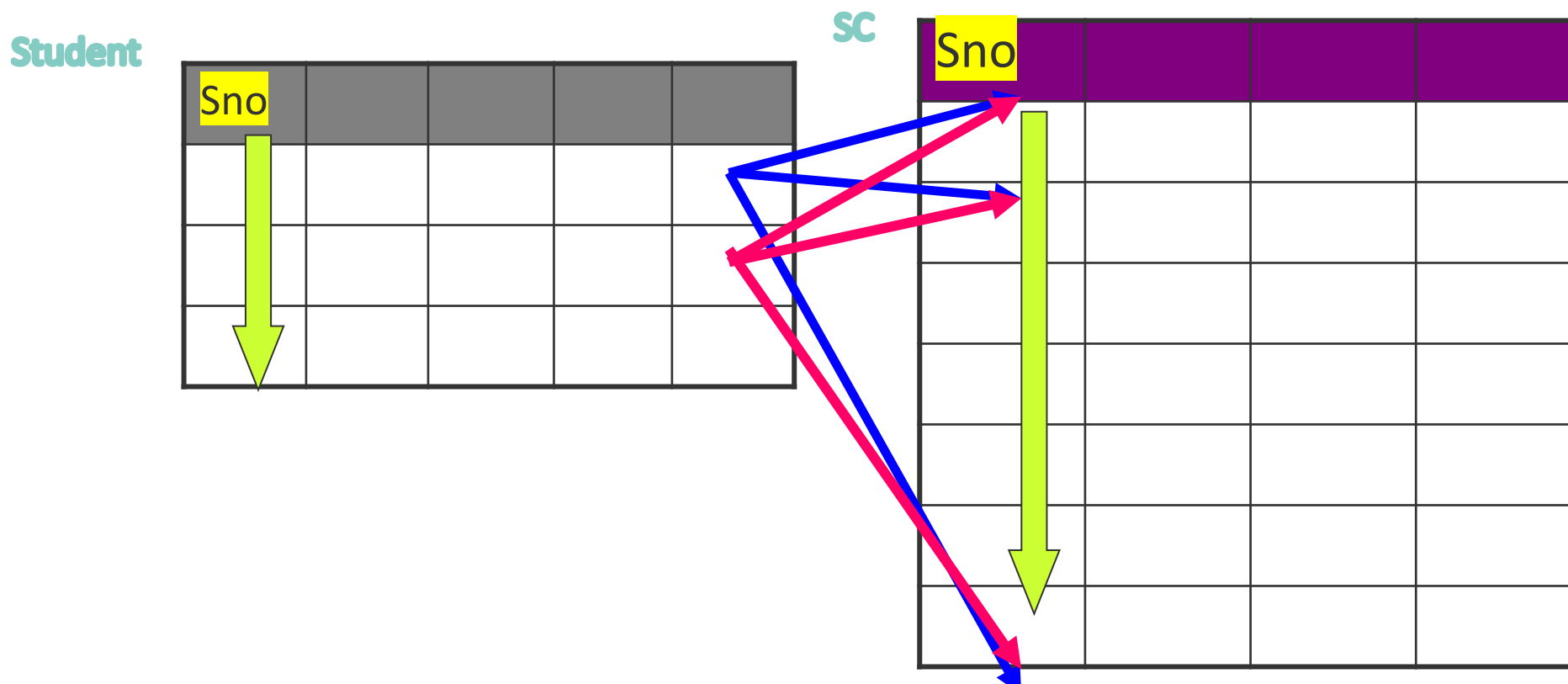
【例】 `SELECT * FROM Student , SC
WHERE Student.Sno=SC.Sno ;`

- 1. 嵌套循环方法(nested loop)
- 2. 排序-合并方法(sort-merge join 或merge join)
- 3. 索引连接(index join)方法
- 4. Hash Join方法



1. 嵌套循环方法(nested loop)

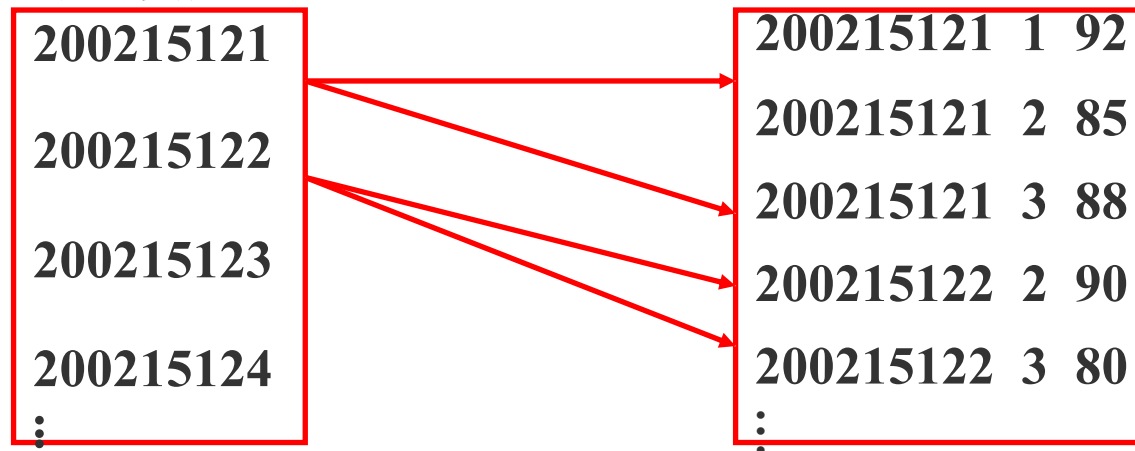
- 对外层循环(Student)的每一个元组(s)，检索内层循环(SC)中的每一个元组(sc)
- 检查这两个元组在连接属性(sno)上是否相等
- 如果满足连接条件，则串接后作为结果输出，直到外层循环表中的元组处理完为止





2. 排序-合并方法(sort-merge join 或merge join)

- 适合连接的表已经排好序的情况
- 排序 - 合并连接方法的步骤：
 - 1.如果连接的表没有排好序，先对Student表和SC表按连接属性Sno排序
 - 2.将排好序的Student表和SC表的数据块读到内容中。
 - 3.取Student表中第一个Sno，依次扫描SC表中具有相同Sno的元组
 - 4.当扫描到Sno不相同的第一个SC元组时，返回Student表扫描它的下一个元组，再扫描SC表中具有相同Sno的元组，把它们连接起来
 - 5.重复上述步骤直到Student 表扫描完



排序-合并连接方法示意图



- **Student表和SC表都只要扫描一遍**
- **如果2个表原来无序，执行时间要加上对两个表的排序时间**
- **对于2个大表，先排序后使用sort-merge join方法执行连接，总的
时间一般仍会大大减少**



查询处理和查询优化

3. 索引连接(index join)方法

- 步骤：

- ① 在SC表上建立属性Sno的索引，如果原来没有该索引
 - ② 对Student中每一个元组，由Sno值通过SC的索引查找相应的SC元组
 - ③ 把这些SC元组和Student元组连接起来
- 循环执行②③，直到Student表中的元组处理完为止



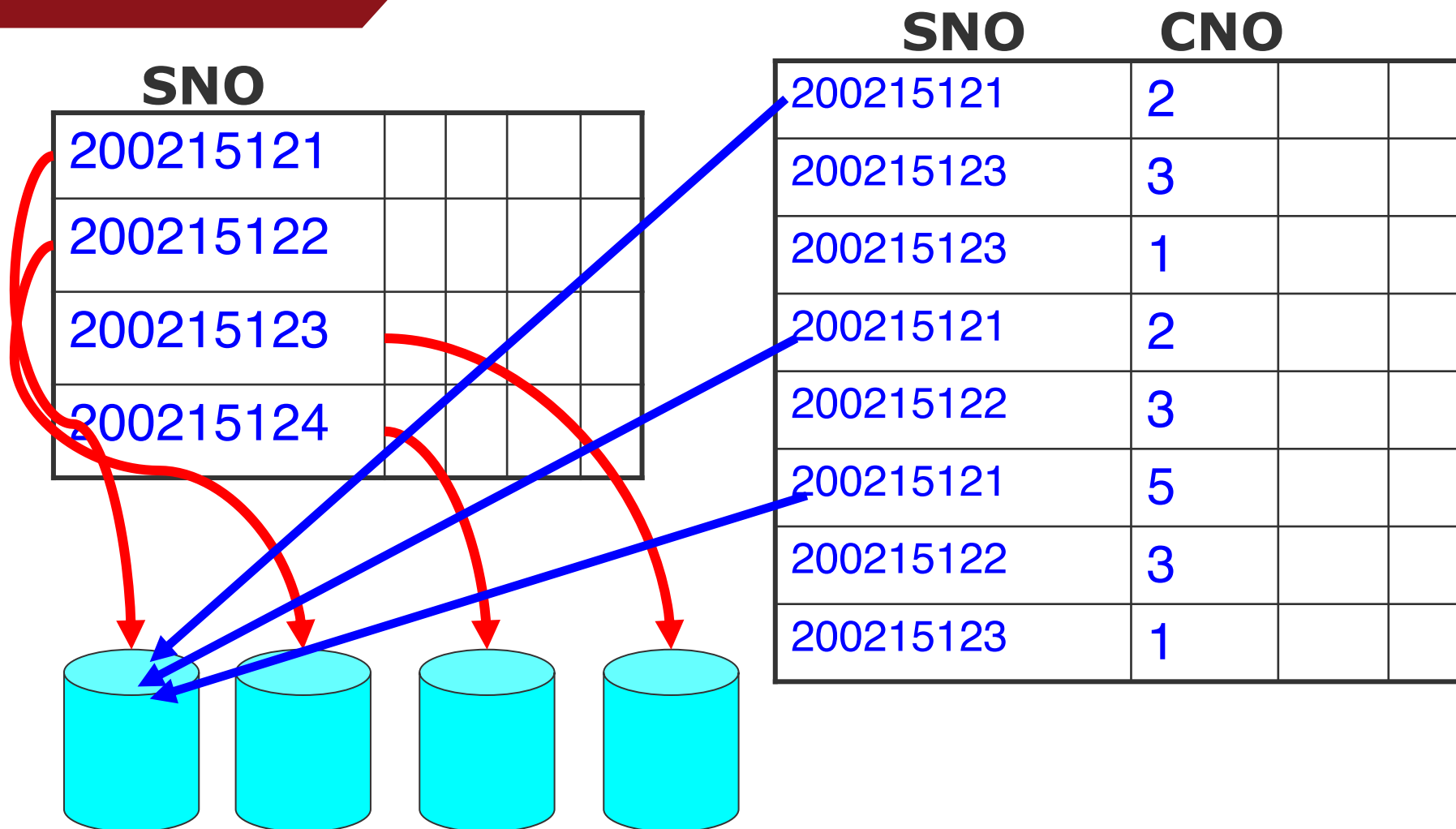


4. Hash Join方法

- 把连接属性作为hash码，用同一个hash函数把Student和SC表中的元组散列到同一个hash文件中
- 步骤：
 - 第一阶段：划分阶段(partitioning phase)：
 - 对包含较少元组的表(比如Student表)进行一遍处理
 - 把它的元组按hash函数分散到hash表的桶中
 - 第二阶段：试探阶段(probing phase)：也称为连接阶段(join phase)
 - 对另一个表(SC表)进行一遍处理
 - 把SC表的元组散列到适当的hash桶中，检索该桶中student表元组。
 - 把SC表元组与桶中所有来自Student表并与之相匹配的元组连接起来



查询处理和查询优化



- hash join算法前提：假设两个表中较小的表在第一阶段后可以完全放入内存的hash桶中
- 以上的算法思想可以推广到更加一般的多个表的连接算法上



查询处理和查询优化



9.0 概述

9.1 查询处理

9.2 查询优化

9.3 代数优化

9.4 物理优化

9.5 小结



查询优化

- 查询优化在关系数据库系统中有着非常重要的地位
- 关系查询优化是影响关系数据库管理系统性能的关键因素
- 由于**关系表达式**的语义级别很高，使关系系统可以从关系表达式中分析查询语义，提供了执行查询优化的可能性



查询优化

- 查询优化的优点不仅在于用户不必考虑如何最好地表达查询以获得较好的效率，而且在于系统可以比用户程序的“优化”做得更好
 - (1) 优化器可以从数据字典中获取许多统计信息，而用户程序则难以获得这些信息
 - (2) 如果数据库的物理统计信息改变了，系统可以自动对查询重新优化以选择适应的执行计划。在非关系系统中必须重写程序，而重写程序在实际应用中往往是不太可能的。
 - (3) 优化器可以考虑数百种不同的执行计划，程序员一般只能考虑有限的几种。
 - (4) 优化器中包括了很多复杂的优化技术，这些优化技术往往只有最好的程序员才能掌握。系统的自动优化相当于使得所有人都拥有这些优化技术



查询处理和查询优化



通过某种代价模型计算出各种查询执行方案的执行代价，然后选取代价最小的执行。

在集中式数据库中

总代价=I/O代价 + CPU代价+内存代价

在分布式数据库中

总代价=I/O代价 + CPU代价+ 内存代价 + 通信代价

由于I/O设计到机械操作，需要的时间比其他高很多，因此在计算代价时一般用查询处理读写的块数作为衡量单位。

查询优化的总目标：

选择有效的策略，求得给定关系表达式的值，使得查询代价最小



查询处理和查询优化



【例】求选修了课程号为2的学生的姓名

```
SELECT Student.Sname  
FROM Student, SC  
WHERE Student.Sno=SC.Sno AND SC.Cno='2';
```

假设 : Student:1000条学生记录,SC:10000条学生记录,
选修2号课程:50条



执行策略

$Q1 = \pi_{Sname}(\sigma_{Student.Sno=SC.Sno \wedge SC.Cno='2'}(Student \times SC))$

$Q2 = \pi_{Sname}(\sigma_{SC.Cno='2'}(Student \bowtie SC))$

$Q3 = \pi_{Sname}(Student \bowtie \sigma_{SC.Cno='2'}(SC))$



执行策略1

$Q1 = \pi_{Sname}(\sigma_{Student.Sno=SC.Sno \wedge SC.Cno='2'}(Student \times SC))$

① Student $\times$ SC

- 在内存中尽可能多的装入某个表（如Student表）的若干块，留出一块存放另一个表（如SC表）的元组
- 把读入的SC表中的每个元组和Student中每个元组连接，连接后的元组装满一块后就写到中间文件上
- 从SC中再读入一块和内存中的Student元组连接，直到SC表处理完
- 再读入若干块Student元组，读入一块SC元组
- 重复上述处理过程，直到把Student表处理完



设一个块能装**10个Student元组**或**100个SC元组**，在内存中存放**5块Student元组**和**1块SC元组**，则

$$\begin{aligned}\text{读取总块数} &= \text{读Student表块数} + \text{读SC表遍数} \times \text{每遍块数} \\ &= 1000/10 + (1000/(10 \times 5)) \times (10000/100) \\ &= 100 + 20 \times 100 = 2100\end{aligned}$$

- 读Student表100块，读SC表20遍，每遍200块，则总计要读取2100数据块
- 连接后的元组数为 $10^3 \times 10^4 = 10^7$ ，设每块能装10个元组，则写出 10^6 块



执行策略1

$$Q1 = \pi_{Sname}(\sigma_{Student.Sno=SC.Sno \wedge SC.Cno='2'}(Student \times SC))$$

② $\sigma_{Student.Sno=SC.Sno \wedge SC.Cno='2'}$

- 依次读入连接后的元组，按照选择条件选取满足要求的记录
- 假定内存处理时间忽略。读取中间文件花费的时间(同写中间文件一样)需读入 10^6 块。
- 若满足条件的元组假设仅50个，均可放在内存。



执行策略1

$Q1 = \pi_{Sname}(\sigma_{Student.Sno=SC.Sno \wedge SC.Cno='2'}(Student \times SC))$

③ π_{Sname}

把第②步的结果在Sname上作投影输出，得到最终结果

策略1执行查询的总读写数据块 = $2100 + 10^6 + 10^6$



执行策略2

$Q2 = \pi_{S.name}(\sigma_{SC.Cno='2'}(Student \bowtie SC))$

① Student $\bowtie$ SC

- 执行自然连接，读取Student和SC表的策略不变，总的读取块数仍为2100块
- 自然连接的结果比第一种情况大大减少，假设为 10^4 个元组
- 写出数据块 = 10^3 块



执行策略2

$Q2 = \pi_{S\ name}(\sigma_{SC.Cno='2'}(Student \bowtie SC))$

② $\sigma_{SC.Cno='2'}$

➤ 读取中间文件块，执行选择运算，读取的数据块 = 10^3 块

③ $\pi_{S\ name}$

➤ 把第2步结果投影输出。

策略2执行查询的总读写数据块 = $2100 + 10^3 + 10^3$

其执行代价大约是第一种情况的488分之一



执行策略3

$Q3 = \pi_{Sname}(Student \bowtie \sigma_{SC.Cno='2'}(SC))$

① $\sigma_{SC.Cno='2'}(SC)$

➤ 先对SC表作选择运算，**只需读一遍SC表，存取100块**，因为满足条件的元组仅50个，不必使用中间文件。

② $Student \bowtie$ ①

➤ 读取Student表，把读入的Student元组和内存中的SC元组作连接。**也只需读一遍Student表共100块。**

③ π_{Sname}

➤ 把连接结果投影输出

策略3总的读写数据块=100+100

其执行代价大约是第一种情况的万分之一，是第二种情况的20分之一



【分析】

假如SC表的Cno字段上有索引

- 第一步就不必读取所有的SC元组而只需读取Cno= '2' 的那些元组(50个)
- 存取的索引块和SC中满足条件的数据块大约总共3 ~ 4 块

若Student表在Sno上也有索引

- 不必读取所有的Student元组
- 因为满足条件的SC记录仅50个，涉及最多50个 Student记录
- 读取Student表的块数也可大大减少



查询处理和查询优化



把代数表达式Q1变换为Q2、 Q3 ,

$$Q1 = \pi_{Sname}(\sigma_{Student.Sno=SC.Sno \wedge SC.Cno='2'}(Student \times SC))$$



$$Q2 = \pi_{Sname}(\sigma_{SC.Cno='2'}(Student \bowtie SC))$$

$$Q3 = \pi_{Sname}(Student \bowtie \sigma_{SC.Cno='2'}(SC))$$

即有选择和连接操作时，**先做选择操作**，这样参加连接的元组就可以大大减少，
这是**代数优化**

在Q3中，SC表的选择操作算法有全表扫描和索引扫描2种方法，经过初步估算，索引扫描方法较优

对于Student和SC表的连接，**利用Student表上的索引**，采用index join代价也较小，这就是**物理优化**



查询处理和查询优化



9.0 概述

9.1 查询处理

9.2 查询优化

9.3 代数优化

9.4 物理优化

9.5 小结



- 代数优化对查询进行等效变换，以减少执行开销。
- 代数优化的原则是**尽量减小查询过程中间结果的大小**。
- 选择、投影操作通常能够有效地减小关系的大小。
- 连接、笛卡尔乘积和并操作容易生成较大的查询中间结果。

因此，先做**选择、投影**；先做**小关系间的连接**，再做**大关系的连接**；甚至需要先找出查询中的公共表达式，以避免重复运算。



- **代数优化策略：通过对关系代数表达式的等价变换来提高查询效率**
- **关系代数表达式的等价：指用相同的关系代替两个表达式中相应的关系所得到的结果是相同的**
- **两个关系表达式E1和E2是等价的，可记为 $E1 \equiv E2$**



常用的等价变换规则

设E1、E2等是关系代数表达式，F是条件表达式

1、连接、笛卡尔积交换律

- $E1 \times E2 \equiv E2 \times E1$
- $E1 \bowtie E2 \equiv E2 \bowtie E1$
- $E1 \bowtie_F E2 \equiv E2 \bowtie_F E1$



2. 连接、笛卡尔积的结合律

$$(E1 \times E2) \times E3 \equiv E1 \times (E2 \times E3)$$

$$(E1 \bowtie E2) \bowtie E3 \equiv E1 \bowtie (E2 \bowtie E3)$$

$$(E1 \underset{F}{\bowtie} E2) \underset{F}{\bowtie} E3 \equiv E1 \underset{F}{\bowtie} (E2 \underset{F}{\bowtie} E3)$$



3. 投影的串接定律

$$\pi_{A_1, A_2, \dots, A_n}(\pi_{B_1, B_2, \dots, B_m}(E)) \equiv \pi_{A_1, A_2, \dots, A_n}(E)$$

假设：

1) E 是关系代数表达式

2) $A_i (i=1, 2, \dots, n)$, $B_j (j=1, 2, \dots, m)$ 是属性名

3) $\{A_1, A_2, \dots, A_n\}$ 是 $\{B_1, B_2, \dots, B_m\}$ 的子集



4. 选择的串接定律

$$\sigma_{F1} (\sigma_{F2} (E)) \equiv \sigma_{F1 \wedge F2}(E)$$

- 选择的串接律说明选择条件可以合并
- 这样一次就可检查全部条件



5. 选择与投影操作的交换律

(1)假设: 选择条件**F**只涉及属性**A1 , ... , An**

$$\sigma_F (\pi_{A1,A2, \dots, An}(E)) \equiv \pi_{A1,A2, \dots, An}(\sigma_F(E))$$

(2)假设: **F**中有不属于**A1, ...,An**的属性**B1,...,Bm**

$$\pi_{A1,A2, \dots, An} (\sigma_F (E)) \equiv \pi_{A1,A2, \dots, An}(\sigma_F (\pi_{A1,A2, \dots, An,B1,B2, \dots, Bm}(E)))$$



6. 选择与笛卡尔积的交换律

(1) 假设：**F中涉及的属性都是E1中的属性**

$$\sigma_F (E1 \times E2) \equiv \sigma_F (E1) \times E2$$

(2) 假设： $F = F1 \wedge F2$ ，并且**F1只涉及E1中的属性**，**F2只涉及E2中的属性**

则由上面的等价变换规则1，4，6可推出：

$$\sigma_F (E1 \times E2) \equiv \sigma_{F1} (E1) \times \sigma_{F2} (E2)$$

(3) 假设： $F = F1 \wedge F2$ ，并且**F1只涉及E1中的属性**，**F2只涉及E1和E2两者的属性**

$$\sigma_F (E1 \times E2) \equiv \sigma_{F2} (\sigma_{F1} (E1)) \times E2$$



■ 7. 选择与并的分配律

设 $E=E_1 \cup E_2$, E_1 , E_2 有相同的属性名, 则

$$\sigma_F(E_1 \cup E_2) \equiv \sigma_F(E_1) \cup \sigma_F(E_2)$$

■ 8. 选择与差运算的分配律

若 E_1 与 E_2 有相同的属性名, 则

$$\sigma_F(E_1 - E_2) \equiv \sigma_F(E_1) - \sigma_F(E_2)$$

■ 9. 选择对自然连接的分配律

$\sigma_F(E_1 \bowtie E_2) \equiv \sigma_F(E_1) \bowtie \sigma_F(E_2)$ F 只涉及 E_1 与 E_2 的公共属性

■ 10. 投影运算与笛卡尔积运算的分配律

■ 11. 投影与并的分配律



■ 10. 投影运算与笛卡尔积运算的分配律

$$\Pi_{A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m}(E_1 \times E_2) \equiv \Pi_{A_1, A_2, \dots, A_n}(E_1) \times \Pi_{B_1, B_2, \dots, B_m}(E_2)$$

- $A_{\{1\}}, A_{\{2\}}, \dots, A_{\{n\}}$ 是 $E_{\{1\}}$ 的属性
- $B_{\{1\}}, B_{\{2\}}, \dots, B_{\{m\}}$ 是 $E_{\{2\}}$ 的属性

■ 11. 投影与并的分配律

若 E_1 与 E_2 有相同的属性名，则

$$\Pi_{A_1, A_2, \dots, A_n}(E_1 \cup E_2) \equiv \Pi_{A_1, A_2, \dots, A_n}(E_1) \cup \Pi_{A_1, A_2, \dots, A_n}(E_2)$$



■典型的启发式规则：

- 1. 选择运算应尽可能先做。在优化策略中这是最重要、最基本的一条

常可使执行时间降低几个数量级，因为选择运算一般使计算的中间结果大大变小。减少中间结果的规模

- 2. 把投影运算和选择运算同时进行

如有若干投影和选择运算，并且它们都对同一个关系操作，则可以在扫描此关系的同时完成所有的这些运算以避免重复扫描关系。避免重复扫描

- 3. 把投影同其前或其后的双目运算（笛卡尔积、连接等）结合起来，没有必要为了去掉某些字段而扫描一遍关系。避免重复扫描



■典型的启发式规则：

- 4. 把某些选择同在它前面要执行的笛卡尔积结合起来成为一个连接运算。

连接特别是等值连接运算要比同样关系上的笛卡尔积省很多时间。**减少中间结果的规模**

- 5. 找出公共子表达式

如果这种重复出现的子表达式的结果不是很大的关系，并且从外存中读入这个关系比计算该子表达式的时间少得多，则先计算一次公共子表达式并把结果写入中间文件。**保存计算结果，避免重复计算。**

例：当查询的是视图时，定义视图的表达式就是公共子表达式的情况。



■ 优化算法

算法：关系表达式的优化

输入：一个关系表达式的语法树。

输出：优化的语法树。

方法：

(1) 分解选择运算

利用规则4(选择的串接)把形如 $\sigma_{F_1 \wedge F_2 \wedge \dots \wedge F_n}(E)$ 变换为 $\sigma_{F_1}(\sigma_{F_2}(\dots(\sigma_{F_n}(E))\dots))$

(2) 通过交换选择运算，将其尽可能移到树的叶端

(3) 通过交换投影运算，将其尽可能移到叶端

注意：

等价变换规则3使一些投影消失

规则5把一个投影分裂为两个，其中一个有可能被移向树的叶端



■ 优化算法

(4) 合并串接的选择和投影，以便能同时执行或在一次扫描中完成

- 利用规则3 ~ 5把选择和投影的串接合并成单个选择、单个投影或一个选择后跟一个投影。
- 使多个选择或投影能同时执行，或在一次扫描中全部完成。
- 尽管这种变换似乎违背“投影尽可能早做”的原则，但这样做效率更高。



(5) 对内结点分组

每一双目运算($\bowtie$, $\times$, $\cup$, $-$)和它所有的直接祖先为一组(这些直接祖先是 σ , π 运算)。

如果其后代直到叶子全是单目运算, 则也将它们并入该组, 但当双目运算是笛卡尔积($\times$), 而且后面不是与它组成等值连接的选择时, 则不能把选择与这个双目运算组成同一组, 把这些单目运算单独分为一组。

(6) 生成程序

生成一个程序, 每组结点的计算是程序中的一步。

各步的顺序是任意的, 只要保证任何一组的计算不会在它的后代组之前计算。



优化的一般步骤

1. 把查询转换成某种内部表示
2. 代数优化：把语法树转换成标准（优化）形式
3. 物理优化：选择底层的存取路径
4. 生成查询计划，选择代价最小的



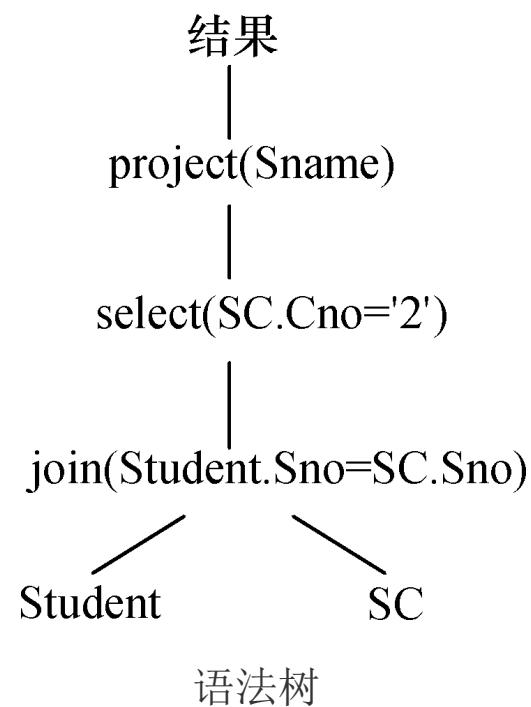
例：求选修了课程号为2的学生姓名

```
SELECT Student.Sname
FROM   Student, SC
WHERE  Student.Sno=SC.Sno
AND    SC.Cno='2';
```

(1) 把查询转换成某种内部表示 (语法树)

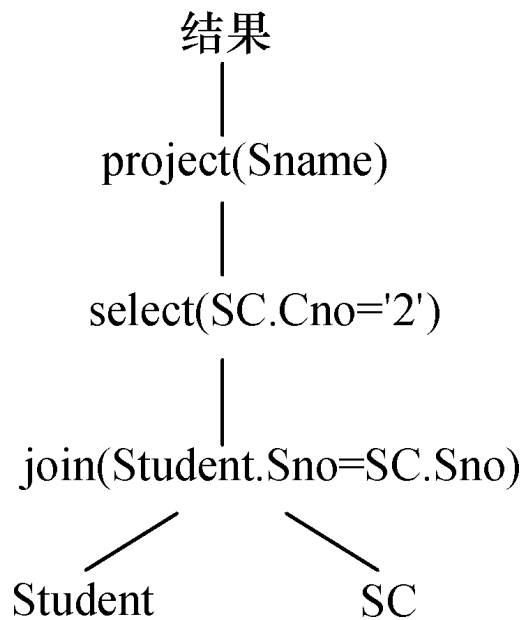
建立语法树的规则：

对一个关系表达式进行语法分析，将关系作为叶子节点，而对关系的操作作为非叶子节点。

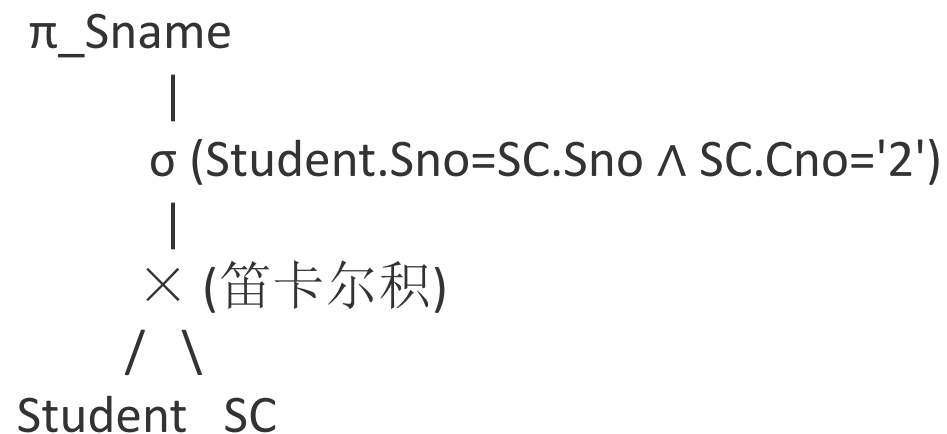




(2) 为了利用优化算法，把语法树转换成标准（优化）形式



语法树



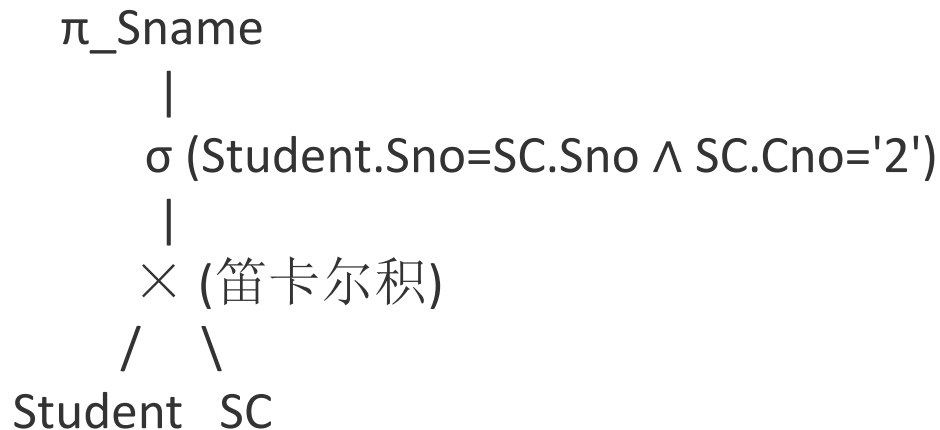
关系代数语法树

$$\pi_{\text{Sname}} (\sigma_{\text{Student.Sno}=\text{SC.Sno} \wedge \text{SC.Cno}='2'}(\text{Student} \times \text{SC}))$$



(3) 对查询树进行优化

利用规则4、6把选择 $\sigma_{SC.Cno='2'}$ 移到叶端，查询树便转换成下图所示的优化的查询树。这就是上节中Q3的查询树表示



关系代数语法树

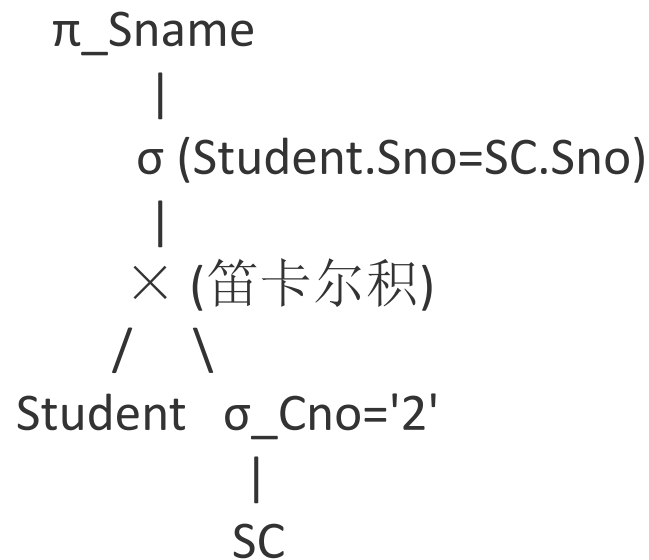
$\pi_{Sname} (\sigma_{Student.Sno=SC.Sno \wedge SC.Cno='2'} (Student \times SC))$

规则4

$\pi_{Sname} (\sigma_{Student.Sno=SC.Sno} (\sigma_{SC.Cno='2'} (Student \times SC)))$

规则6

$\pi_{Sname} (\sigma_{Student.Sno=SC.Sno} (Student \times \sigma_{Cno='2'} (SC)))$

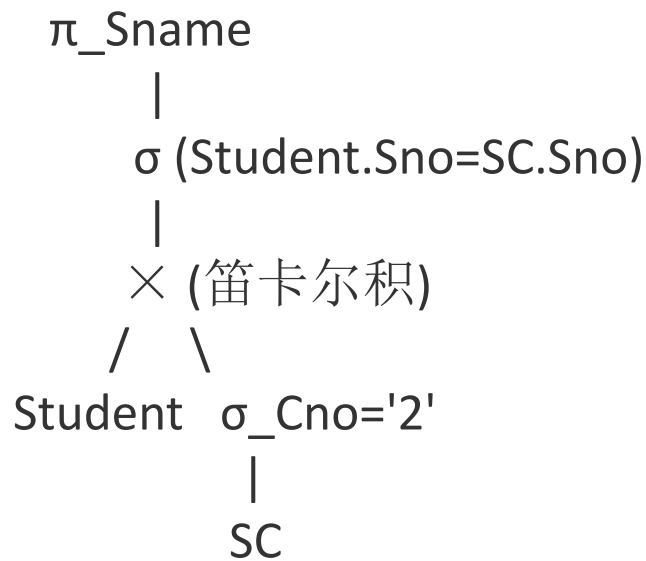


关系代数语法树

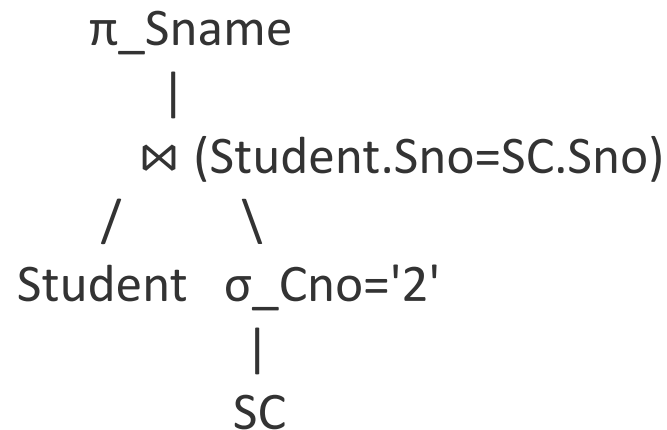


(3) 对查询树进行优化

利用规则4、6把选择 $\sigma_{SC.Cno='2'}$ 移到叶端，查询树便转换成下图所示的优化的查询树。这就是上节中Q3的查询树表示



关系代数语法树



优化后的语法树

$\pi_{Sname} (\sigma_{Student.Sno=SC.Sno} (Student \times \sigma_{Cno='2'}(SC)))$

$\pi_{Sname} (Student \bowtie_{Student.Sno=SC.Sno} \sigma_{Cno='2'}(SC))$



查询处理和查询优化



9.0 概述

9.1 查询处理

9.2 查询优化

9.3 代数优化

9.4 物理优化

9.5 小结



- **代数优化改变查询语句中操作的次序和组合，不涉及底层的存取路径**
- **对于一个查询语句有许多存取方案，它们的执行效率不同，仅仅进行代数优化是不够的**
- **物理优化就是要选择高效合理的操作算法或存取路径，求得优化的查询计划**



■ 选择的方法：

基于规则的启发式优化

启发式规则是指那些在大多数情况都适用，但在所有情况下都是最好的规则

基于代价估算的优化

优化器估算不同执行策略的代价，并选出具有最小代价的查询计划

两者结合的优化方法: 常常先用启发式规则，选取若干较优的候选方案，减少代价估算的工作量；然后分别计算这些候选方案的执行代价，较快地选出最终的优化方案。



- 一、选择操作的启发式规则：
 - 对于小关系，使用全表顺序扫描，即使选择列上有索引
 - 对于大关系，启发式规则有：
 - 对于选择条件是主码 = 值的查询
 - 查询结果最多是一个元组，可以选择主码索引
 - 一般的RDBMS会自动建立主码索引。
 - 对于选择条件是非主属性 = 值的查询，并且选择列上有索引
 - ✓ 要估算查询结果的元组数目
 - 如果比例较小(<10%)可以使用索引扫描方法
 - 否则还是使用全表顺序扫描



■ 一、选择操作的启发式规则:

- 对于选择条件是属性上的**非等值查询或者范围查询**，并且选择列上有索引
 - ✓ 要估算查询结果的元组数目
 - 如果比例较小($<10\%$)可以使用索引扫描方法
 - 否则还是使用全表顺序扫描
- 对于用**AND连接**的合取选择条件
 - ✓ 如果有涉及这些属性的组合索引
 - 优先采用组合索引扫描方法
 - ✓ 如果某些属性上有一般的索引
 - 可以使用索引扫描，其他情况使用全表顺序扫描。
- 对于用**OR连接**的析取选择条件，一般使用全表顺序扫描



- **二、 连接操作的启发式规则：**

1. 如果2个表都已经按照连接属性排序

 选用排序-合并方法

2. 如果一个表在连接属性上有索引

 选用索引连接方法

3. 如果上面2个规则都不适用，其中一个表较小

 选用Hash join方法

4. 可以选用嵌套循环方法，并选择其中**较小的表**，确切地讲是占用的块数(b)较少的表，作为**外表**(外循环的表)。

 设连接表R和S分别占用的块数为 B_r 与 B_s ，连接操作使用的内存缓冲区块数为 K ，分配 $K-1$ 块给外表，如果R为外表，则嵌套循环法存取块数为 $B_r + B_r B_s / (K-1)$ ，显然改选块数小的表作为外表



- 启发式规则优化是定性的选择，适合解释执行的系统
 - 解释执行的系统，优化开销包含在查询总开销之中
- 编译执行的系统中查询优化和查询执行是分开的
 - 可以采用精细复杂一些的**基于代价的优化方法**
 - 根据数据字典中的统计信息
 - 计算各种操作算法的执行代价，从中选择最小的



基于代价的优化

- 1.统计信息
- 2.代价估算示例
- 3.优化方法



1.统计信息

基于代价的优化方法要计算查询的各种不同执行方案的执行代价，它与数据库的状态密切相关

优化器需要的统计信息

(1) 对每个基本表

- **该表的元组总数(N)**
- **元组长度(l)**
- **占用的块数(B)**
- **占用的溢出块数(BO)**



(2) 对基表的每个列

- 该列不同值的个数(m)
- 列最大值
- 最小值
- 列上是否已经建立了索引
- 哪种索引(B+树索引、Hash索引、聚集索引)
- 可以计算选择率(f)
 - 如果不同值的分布是均匀的, $f = 1/m$
 - 如果不同值的分布不均匀, 则要计算每个值的选择率, $f = \text{具有该值的元组数}/N$



(3) 对索引

- 索引的层数(L)
- 不同索引值的个数
- 索引的选择基数S(有S个元组具有某个索引值)
- 索引的叶结点数(Y)



2.代价估算示例

(1) 全表扫描算法的代价估算公式

- 如果基本表大小为B块，全表扫描算法的代价 $\text{cost} = B$
- 如果选择条件是“码 = 值”，那么平均搜索代价 $\text{cost} = B/2$



2.代价估算示例

(2) 索引扫描算法的代价估算公式

如果选择条件是“码 = 值”

- 则采用该表的主索引
- 若为B+树，层数为L，需要存取B+树中从根结点到叶结点L块，再加上基本表中该元组所在的那一块，所以 $\text{cost} = L + 1$



2.代价估算示例

(2) 索引扫描算法的代价估算公式

如果选择条件涉及非码属性

- 若为B+树索引，选择条件是相等比较，S是索引的选择基数(有S个元组满足条件)
- 满足条件的元组可能会保存在不同的块上，所以(最坏的情况) $\text{cost} = L + S$



2.代价估算示例

(2) 索引扫描算法的代价估算公式

如果比较条件是 $>$, $> =$, $<$, $< =$ 操作

- 假设有一半的元组满足条件
- 就要存取一半的叶结点
- 通过索引访问一半的表存储块
- $\text{cost} = L + Y/2 + B/2$
- 如果可以获得更准确的选择基数，可以进一步修正Y/2与B/2



2.代价估算示例

(3) 嵌套循环连接算法的代价估算公式

嵌套循环连接算法的代价

$$\text{cost} = B_r + B_r B_s / (K - 1)$$

如果需要把连接结果写回磁盘

$$\text{cost} = B_r + B_r B_s / (K - 1) + (F_{rs} * N_r * N_s) / M_{rs}$$

- 其中 F_{rs} 为连接选择性(join selectivity), 表示连接结果元组数的比例
- M_{rs} 是存放连接结果的块因子, 表示每块中可以存放的结果元组数目



2.代价估算示例

(4) 排序-合并连接算法的代价估算公式

如果连接表已经按照连接属性排好序，则

$$\text{cost} = B_r + B_s + (F_{rs} * N_r * N_s) / M_{rs}$$

如果必须对文件排序

- 还需要在代价函数中加上排序的代价



查询处理和查询优化



9.0 概述

9.1 查询处理

9.2 查询优化

9.3 代数优化

9.4 物理优化

9.5 小结



- **查询处理是RDBMS的核心，查询优化技术是查询处理的关键技术**
- **本章讲解的优化方法**
 - 基于关系代数等价变换规则的代数优化
 - 基于启发式规则的存取路径优化
 - 基于代价的优化
- **本章的目的：掌握查询优化方法的概念和技术，尤其是涉及连接和嵌套的查询**
 - 不要把优化的任务全部放在RDBMS上，应该找出RDBMS的优化规律，以写出适合RDBMS自动优化的SQL语句
- **对于RDBMS不能优化的查询需要重写查询语句，进行手工调整以优化性能**